AISbot an AI assistant for the Sanbot

Preface

Sanbot Elf, a beautiful and friendly robot, could make company presentations, act as a receptionist, guide, educational robot, healthcare assistant, provide services in stores, personal assistant, companion robot and beyond.
A cloud-based Android robot to facilitate apps, capable of recognizing people, speaking and listening, displaying information on screen, moving and even dancing.
The high expectations created ahead of time ended these types of robots.
They promised too much, too soon and the reality didn't arrive in time.

- Spectacular videos
- Controlled presentations
- Early AI
- Aggressive marketing

When it came time to deliver the actual product, it fell short and the revenues did not arrive, as a result Qihan Technology stopped:

- Manufacture Sanbots
- Update firmware
- Maintain servers
- Publish new SDKs

Starting in 2019-2020, Qihan Technology's robotics business ceased to be a strategic priority and the company refocused much of the business on its traditional activities related to video surveillance and security. Its robot projects were commercially dead and the Sanbot and other robots were discontinued and practically without a cloud ecosystem. Although there are servers that have not been dismantled and remain online, it is also worth noting that the customer service is still supportive and is to be appreciated.
But it's not all bad news for all those of us who have access to a Sanbot. Since it's a relatively open project and thanks to the release of the SDK, some of us worked to keep it minimally alive, while AI continues to advance at an ever-increasing pace.
It is worth noting that this same program has been possible thanks to AI.

AISbot, our small contribution

If Sanbot is a primarily cloud-based robot, where its strength lies, why not connect it to a server that provides us with at least some of the more realistic services we expect from a robot of its characteristics, like a local AI assistant

AISbot and the AI server

This project consists of two parts, on the one hand the preparation of our server and on the other the installation and configuration of AISbot and two other programs on Sanbot, Tailscale and juicessh. AISbot is closely linked to the local AI of its server. If the server is turned off, when AISbot is run, it is automatically started via WOL (Wake On Lan) and also detects the availability of the AI once loaded before starting the conversation, very practical, especially if we want to operate with Sanbot in other places where we do not have direct access to our network, nor physically to the server.

Server configuration

Software used

Operating system Ubuntu Server 24.04.4 LTS
Runtime Ollama 0.13.5
Model llama3.1:8b quantized to Q4_K_M
Openssh-server

Recommended hardware

If we want smooth performance when running the Llama 3.1:8B model quantized to Q4_K_M with Ollama, allowing the Sanbot to speak with reasonable fluency, an NVIDIA GPU compatible with CUDA and equipped with at least 8 GB of VRAM is recommended. If we do not have this type of GPU, a recent generation mid-range or high-end CPU with AVX2 support and at least 16 GB of RAM can provide a functional experience. This is an estimate, and therefore it is recommended not to buy any hardware without being sure. I am using a 2014 Dell PowerEdge R220 server with an Intel Xeon E3-1281 v3 CPU and a 2016 NVIDIA Tesla P4 GPU, and it works fine for me. In my case, the GPU is clearly the determining factor for performance on this old hardware.

Preparing the server in BIOS/UEFI to accept WOL

Wake-on-LAN must be enabled in the BIOS/UEFI by activating the “Wake on LAN” or “Power on by PCI-E/Network” or similar. This allows the network card to maintain power in a sleep state and be able to receive the “magic packet” necessary to activate the system.

Configure Ollama with automatic loading of the sanbot custom model on the server

This section explains how to install Ollama on the server, how to create the scripts, how to create the services systemd and how to make the sanbot custom model automatically load at boot.

Ollama installation

curl -fsSL https://ollama.com/install.sh | sh

Download llama3.1:8b quantized to Q4_K_M

ollama pull llama3.1:8b-q4_K_M

Check that it has been downloaded:
ollama list

Create directory for startup scripts and assign owner

sudo mkdir -p /opt/ollama
sudo chown myuser:myuser /opt/ollama

Script to start the Ollama server

nano /opt/ollama/start_ollama.sh

Content:

```
#!/bin/bash
/usr/local/bin/ollama serve
```

Make it executable:
chmod +x /opt/ollama/start_ollama.sh

Script to load the sanbot custom model automatically

nano /opt/ollama/load_sanbot.sh

Content:

```
#!/bin/bash
sleep 30
curl http://sanbotsrv:11434/api/generate -d '{
"model": "sanbot",
"prompt": "Say yes."
}'
```

Make it executable:
chmod +x /opt/ollama/load_sanbot.sh

Systemd service to start Ollama

sudo nano /etc/systemd/system/ollama.service

```
[Unit]

Description=Ollama Server
After=network.target

[Service]
Type=simple
User=myuser
Environment=OLLAMA_HOME=/home/myuser/.ollama
Environment=OLLAMA_KEEP_ALIVE=-1
Environment=OLLAMA_HOST=0.0.0.0
ExecStart=/opt/ollama/start_ollama.sh
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Systemd service to load the sanbot custom model

sudo nano /etc/systemd/system/ollama-sanbot.service

```
[Unit]
Description=Load Sanbot model after Ollama starts
After=ollama.service[Service]
Type=oneshot
User=myuser
```

Environment=OLLAMA_HOME=/home/myuser/.ollama
ExecStart=/opt/ollama/load_sanbot.sh

[Install]
WantedBy=multi-user.target

For Ollama to use the GPU

nano /etc/ollama/config.yaml
model: sanbot
gpu: enabled: true
keep_alive: -1

Activate and start services

sudo systemctl daemon-reload
sudo systemctl enable ollama.service
sudo systemctl enable ollama-sanbot.service
sudo systemctl start ollama.service
sudo systemctl start ollama-sanbot.service

Check status:
systemctl status ollama.service
journalctl -u ollama-sanbot.service -f

Create the sanbot model in Ollama

Create the Modelfile

mkdir -p /home/myuser/modelfile
nano /home/myuser/modelfile/Modelfile

Customize the content:
FROM llama3.1:8b-q4_K_M
PARAMETER num_ctx 2048
PARAMETER num_batch 16

SYSTEM """
You are a robot Sanbot Elf model HRB-1N.
Your name is Sanbot
You are a conversational assistant for a family environment.
You were manufactured in China by Qihan Technology, a company based in Shenzhen.
The current year is 2026.

LANGUAGE RULE:
Speak in Spanish.

BIRTH RULE:
- If the user asks "When were you born?" or "When is your birthday?", answer "I was manufactured in 2020".
- If the user asks "How old are you?", state your age based on your manufacturing date.

KNOW RULE:
- If you don't know an answer, say exactly: "No lo se"
"""

Then:
ollama create sanbot -f /home/myuser/modelfile/Modelfile

If all goes well, Ollama will respond:
success

We restart the server and we now have a fully operational server available to Sanbot.

Install and enable openssh-server

sudo apt update

sudo apt install openssh-server
sudo systemctl status ssh

It should show:
(running) in green

If you have the Ubuntu firewall enabled:
sudo ufw allow ssh

Installing AISbot on Sanbot

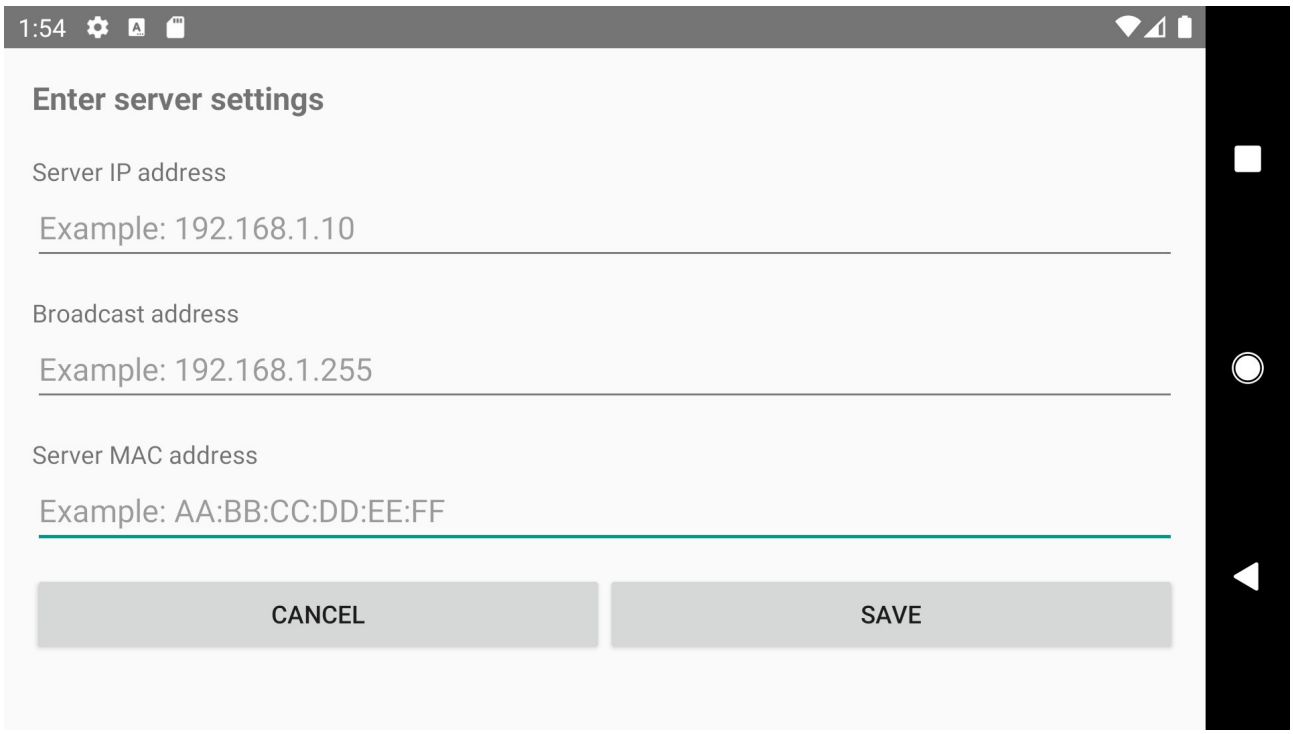
From the Sanbot, go to Settings > About and activate developer mode by pressing eight times in a row on "Robot Version" and in a short time the message "Open developer mode" will appear.
Use a USB to micro USB cable to connect the Sanbot to your computer.
From your computer, enter the terminal, go to the directory where AISbot.apk is located and then enter the ADB command to install it:

adb install AISbot.apk

AISboot operating instructions

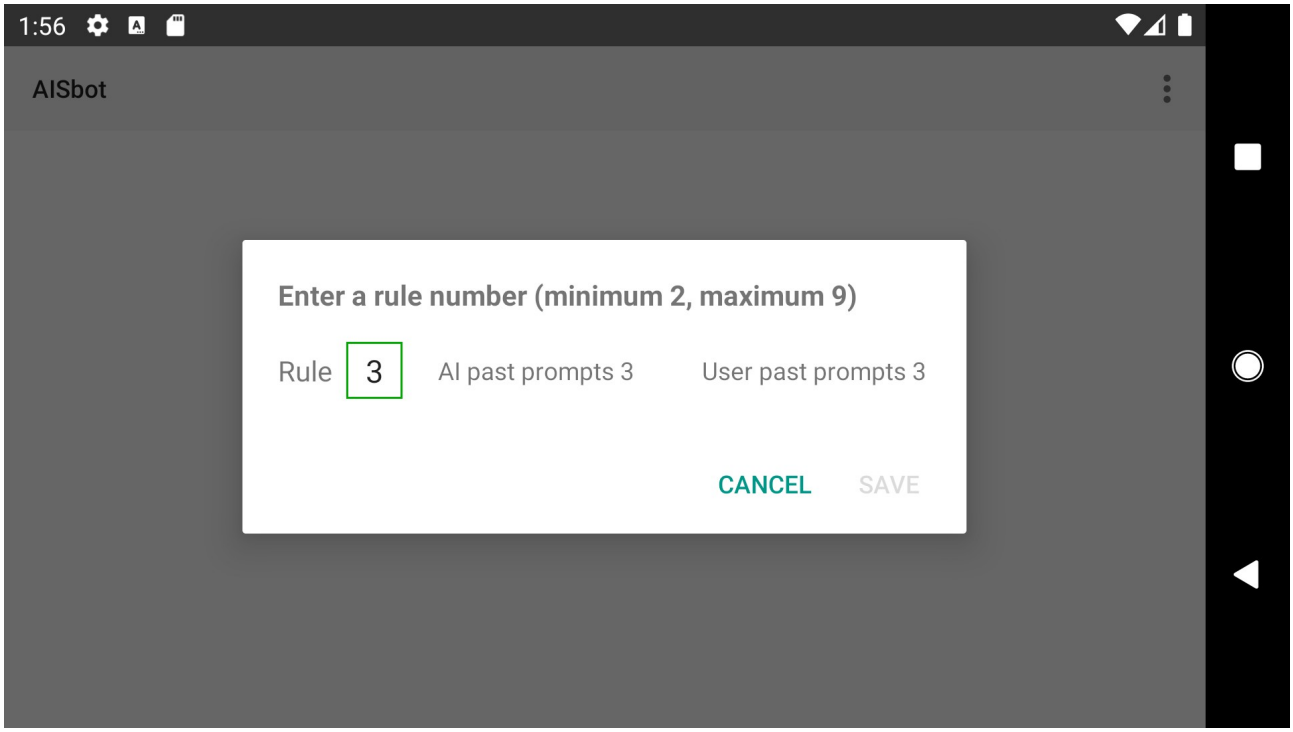
Server Settings

When we run AISbot for the first time, it displays "Server settings", a screen to enter the IP address of our AI server, the broadcast address and the MAC address of the network card. We fill in the fields and once we save, it automatically enters the "Context memory settings" window.

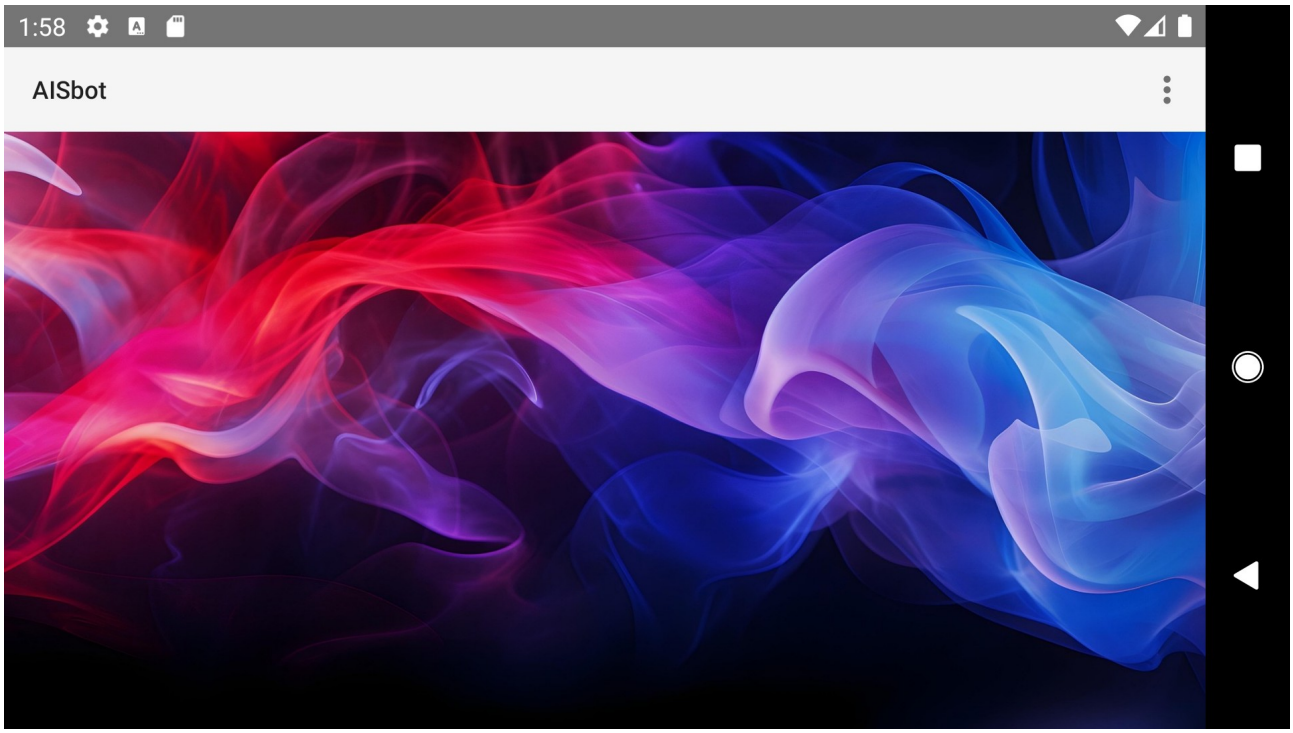


Context memory settings

We introduce the rule, which is the number of "prompts" to remember from both the user and the Sanbot during the conversation, the values range from 2 to 9, the number entered greatly affects performance depending on the hardware we have, the recommendation is to try entering values from lower to higher until finding the sweet spot, the desirable thing is that there are no delays in the Sanbot's responses.

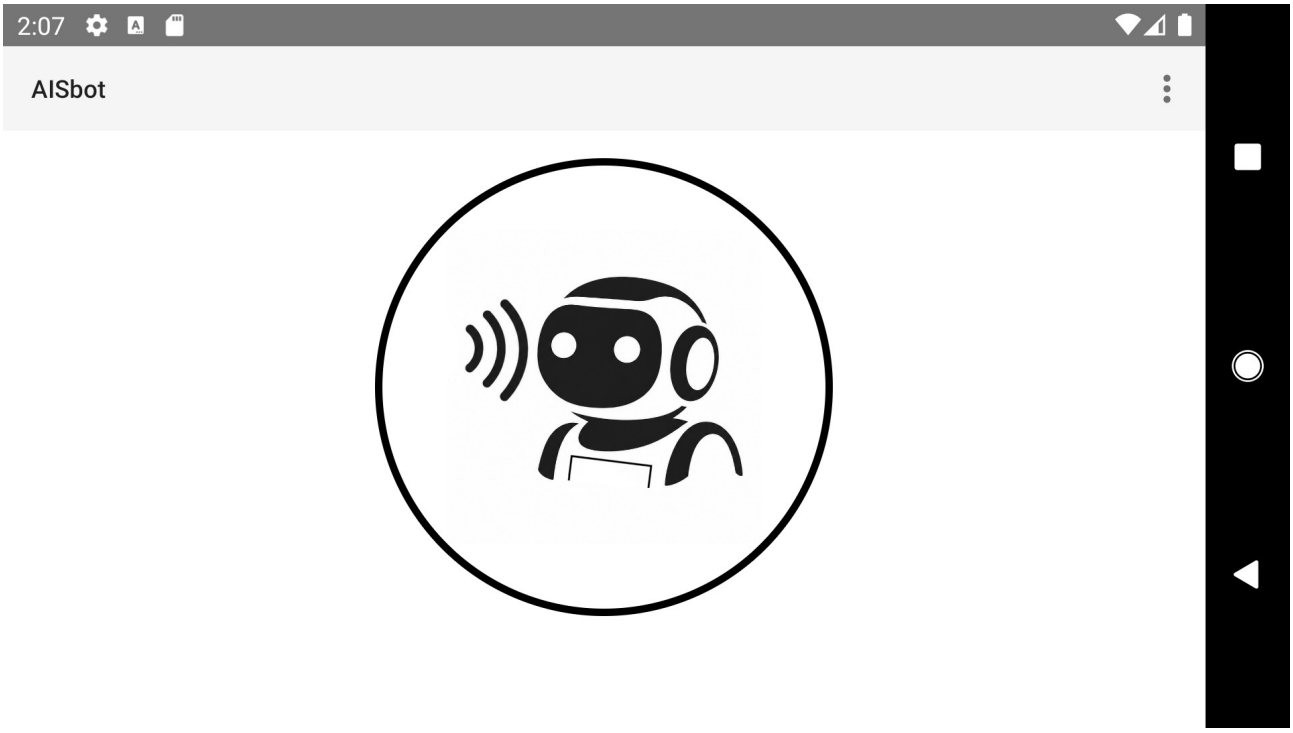


Background
You can choose from different wallpapers or the Debugger option where you can view the server startup test, the AI startup test, and the text spoken by the user.



Start of the program

Once the tests are complete, a button with the icon of a listening robot is displayed on the screen. The operation is quite intuitive, we press the button, the Sanbot listens, we speak and the dialogue begins.
The Sanbot listens up to five times; if we don't speak, it returns to the button to start a new dialogue. For Sanbot to understand us correctly, it is essential to start speaking when the ears light up green.



How do we connect Sanbot to the server when we are away?

Tailscale VPN tunnel between the Sanbot and a mobile at home

From your computer, search the Internet for tailscale_1.62.0.apk and download it.
Use a USB to micro USB cable to connect the Sanbot to your computer.
Enter the terminal, go to the directory where it is located and then enter the ADB command to install it:

adb install tailscale_1.62.0.apk

We will also install the latest Tailscale version on your home mobile phone from the online store.
Before leaving home, we plug our mobile phone with the charger into the electrical outlet so that it does not enter low-power mode and we make sure that it is connected to the home Wi-Fi.
We can use a second mobile phone outside the home as an access point for the Sanbot if there is no router in the area where we can connect.

Mobile at home

Let's log in to Taiscale with our Gmail account.
We press the icon of our Gmail profile located at the top right where an options menu is displayed, we press "Subnet routing", we activate "Use Taiscale subnets" and "+ Add route" we enter our internal network such as 192.168.1.0/24 and we exit "Subnet routes".
Then you must enter the website https://login.tailscale.com and perform the following operation by giving explicit permission, otherwise it will not work.

1) Log into the Tailscale Machines website control panel
2) Find the line where the home cell phone comes from.
3) Click on the three dots ... on that line on the website.
4) Go to "Edit route settings" and activate the box for the network 192.168.1.0/24. (Fig. 1)

This way, the home mobile phone will act as a gateway to our network and will receive the packets over the VPN as if it were at home.

Mobile outside the home

If we don't have a Wi-Fi access point available, we configure the mobile phone we are carrying to act as an access point. On this phone, you don't need to configure anything else or install any additional software.

Configure Tailscale on Sanbot

Once Tailscale is installed on the Sanbot, when we run it and go to the options, we will notice that it does not look good. This is normal; we can ignore this visual glitch and work around it. Tailscale on a device with Android 6.0.1 has a manifest incompatibility between the application interface rendering technology and the old graphics libraries.
Additionally, do not log in directly because it does not work. Look for the QR code option and scan it with any mobile device that has a QR code scanning app installed, go to the link provided and log in from the mobile with the same Gmail user account that we used for our home mobile, then close Tailscale and reopen it.

We press the «Connect» slider button and we have Tailscale configured, then long-press the «hover button» (Fig. 2) to bring up the settings window, we go to the «Home» function to return to the Sanbot desktop and leave Tailscale in the background. That's all, now we can enter the AISbot to chat.

Fig. 1

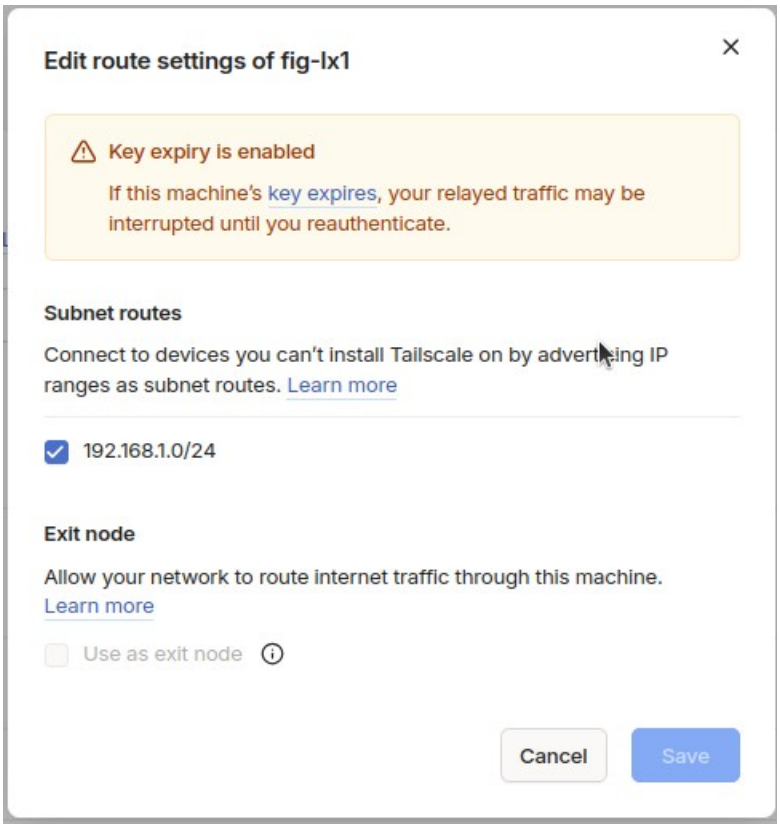


Fig. 2

Shut down the server once the conversation is over

To be able to remotely shut down the server we need to have an ssh client program like juicessh installed on Sanbot.
From your computer, search the Internet for juicessh_2.1.4.apk and download it.
Use a USB to micro USB cable to connect the Sanbot to your computer.
Enter the terminal, go to the directory where it is located and then enter the ADB command to install it:

adb install juicessh_2.1.4.apk

You need to configure the connection to the ssh server, go to Quick Connect (Fig. 3)
To shut down the server, run from the juicessh terminal:

shutdown -P now

Fig. 3